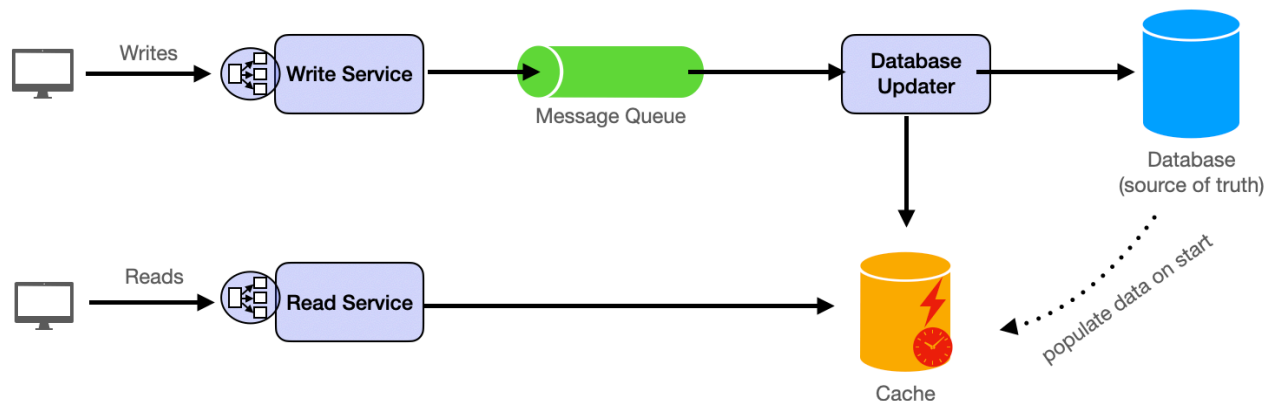


Build a High-Performance Social Media Comment System with Kafka & Redis

 systemdesignschool.io/fundamentals/system-design-template-social-media-comments



System Design Template Application: Social Media Comment System

In this lab, we are going to construct a social media comment application from the ground up, utilizing the essential components we've mastered in previous sections. This exercise will provide a practical understanding of:

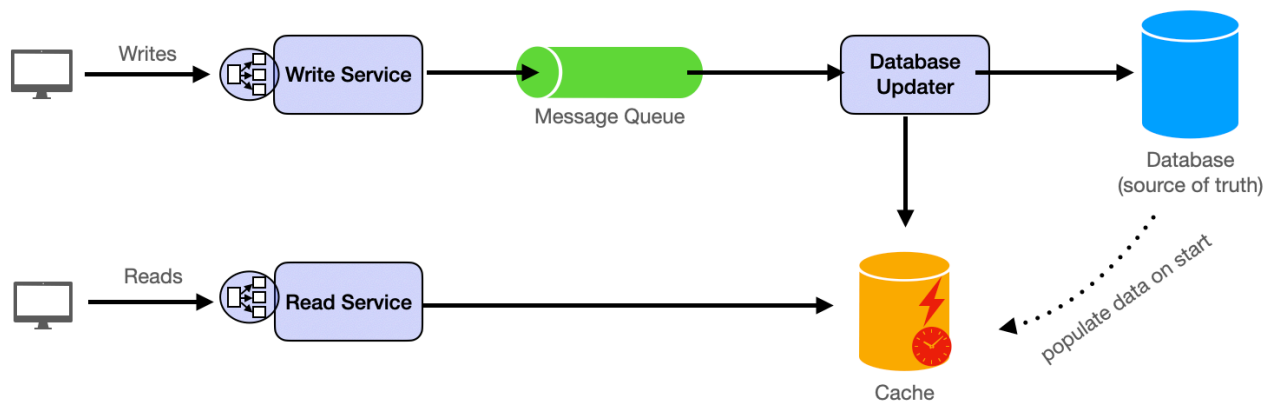
- The role of **message queues as a buffer** in addressing the processing rate disparities between services.
- The concept and application of **eventual consistency**.
- The process of employing the template to design and develop applications with **distinct read and write paths**.

For popular posts, there could be a burst of traffic with a large number of comments. This imposes a large load on the database which has a limited write capacity. We typically use a message queue as a buffer between the fast comment posting and the limited database write capacity.

On the other hand, having millions of users reading the comments imposes an even greater load on the database. We also never have the user-facing services read directly from the database. What we do instead is to cache the data from the database and have the user-facing services read from the cache.

System Design Overview

Our system design diagram looks like this:



System Design Overview

Components:

- Comment Uploader: Exposes an HTTP POST API that allows a user to upload a comment. We will write a simple Flask server to do this.
- Message queue: A buffer between user facing service and database updater. We will use Kafka for this.
- Database Updater: Takes messages from the message queue and write to the database for permanent storage. We will write a simple Kafka consumer in python.
- Database: We can use a NoSQL database for a comment system like this without complex relationships. In this example, we will use CouchDB which has a feature called “changes feed” that acts as triggers that we can use to write to our cache.
- Cache: We will use Redis as our cache.
- Comment Fetcher: A simple service that exposes an HTTP GET API that allows a user to fetch all the comments for a post.

Dataflow

We separate write and read paths.

For the write path:

- User creates a comment and send it to the Comment Uploader using Comment Uploader’s HTTP POST API.
- The Comment Uploader enqueues the comment as a message into the message queue.
- Message queue consumer Database updater fetches the messages, extracts comments and inserts them into the database.
- After the insert is completed, a database trigger is fired and sends an action to write the record into the cache.

For the read path:

Users opens a page with comments which sends an API request to the Comment Fetcher which fetches the comments from the cache and returns them to the client.

Data Schema

docker-compose.yml

This file sets up the infrastructure including message queue (Kafka), database (MongoDB) and cache (Redis) so we can write our application code using them.

```
version: '3' services: kafka: image: wurstmeister/kafka ports: - "9092:9092"
environment: KAFKA_ADVERTISED_HOST_NAME: localhost KAFKA_ZOOKEEPER_CONNECT:
zookeeper:2181 KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
KAFKA_DELETE_TOPIC_ENABLE: 'true' zookeeper: image: wurstmeister/zookeeper
ports: - "2181:2181" redis: image: redis:latest ports: - "6379:6379" mongo:
image: mongo:latest ports: - "27017:27017" volumes: couchdb_data:
```

MongoDB

We'll set up our database as `socialnetwork` and have a `comments` collection with the following schema:

```
{
  "_id": ObjectId,
  "post_id": string,
  "comment": string
}
```

For example:

```
socialnetwork> db.comments.find() [ { _id: ObjectId("6466a681bec0bd23bec4ce12"),
post_id: '1', comment: 'this is a great post thank you' } ]
```

Redis

In Redis, we'll use a Redis hash to store comments using `post_id` as the key:

Key: "post:<post_id>"
Value: List of comments

For example:

```
127.0.0.1:6379> hgetall "post:1" 1) "6466a29c6fb786d66e584e5a" 2) "{\"post_id\":
\\\"1\\\", \\\"comment\\\": \\\"this is a great post thank you\\\", \\\"_id\\\":
\\\"6466a29c6fb786d66e584e5a\\\"}" 3) "6466a681bec0bd23bec4ce12" 4) "{\"post_id\":
\\\"1\\\", \\\"comment\\\": \\\"this is another great post thank you\\\", \\\"_id\\\":
\\\"6466a681bec0bd23bec4ce12\\\"}"
```

Kafka

Our `comment_uploader` will push comments as messages into a topic Kafka topic called `comments`.

Services

Exposes an HTTP POST API that allows a user to upload a comment. We will write a simple Flask server to do this. It returns 200 right after it pushes the message into the queue so users will get an instant response.

```
from flask import Flask, request from kafka import KafkaProducer import json app = Flask(__name__) producer = KafkaProducer(bootstrap_servers='localhost:9092') @app.route('/comments', methods=['POST']) def upload_comment(): data = request.get_json() post_id = data.get('post_id') comment = data.get('comment') producer.send('comments', value=json.dumps({'post_id': post_id, 'comment': comment}).encode()) return 'Comment uploaded', 200 if __name__ == '__main__': app.run(host='0.0.0.0', port=5001)
```

Database Updater (Kafka Consumer)

This is the Kafka consumer that listens for new messages on the `comments` topic. When a new message comes in, it first writes to the database and then update the cache.

```
from kafka import KafkaConsumer from pymongo import MongoClient import redis import json from bson import ObjectId # encode mongodb's ObjectId to be compatible with JSON class JSONEncoder(json.JSONEncoder): def default(self, o): if isinstance(o, ObjectId): return str(o) return json.JSONEncoder.default(self, o) consumer = KafkaConsumer('comments', bootstrap_servers=['localhost:9092'], value_deserializer=lambda x: json.loads(x.decode('utf-8'))) client = MongoClient('mongodb://localhost:27017/') db = client['socialnetwork'] collection = db['comments'] redis_client = redis.Redis(host='localhost', port=6379, db=0) for message in consumer: comment = message.value # insert into database result = collection.insert_one(comment) # update the cache if insertion into database is successful if result.acknowledged: post_id = comment['post_id'] comment_id = str(result.inserted_id) # Store the comment in a hash in Redis. The hash key is the post_id, # and each field in the hash is a comment_id. redis_client.hset(f'post:{post_id}', comment_id, json.dumps(comment, cls=JSONEncoder)) else: print("Error: Failed to insert comment to MongoDB")
```

A web server that provides an API to return comments from the Redis cache.

```
from flask import Flask, request import redis import json app = Flask(__name__) # Create a connection to Redis redis_client = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True) @app.route('/comments', methods=['GET']) def get_comments(): post_id = request.args.get('post_id') # Fetch all comments related to the post_id comments_dict = redis_client.hgetall(f'post:{post_id}') comments = [json.loads(comment) for comment in comments_dict.values()] return {'comments': comments} if __name__ == "__main__": app.run(host="0.0.0.0", port=5002, debug=True)
```

Starting the system

First, let's get the infrastructure up and running using `docker-compose up --build`. This will start Kafka at port 9092, Redis at port 6379 and MongoDB at port 27017. You should see 4 services running; the extra one being [Zookeeper which Kafka depends on for consensus](#).

<

comments

/Users/sheldon/cache/system-design-demos/comments

Open

comments-zookeepe... wurstmeister/zookeepe Running 2181:2181	lance with old generation 0 (__consumer_offsets-34) (reason: Addi consumer-40290bcd-e4ad-499e-ba33-cfcd49ffa90e with group instance inator.group.GroupCoordinator) 2023-05-18 15:08:33 comments-kafka-1 [2023-05-18 22:08:33,902] INFO [GroupCoordi nator 1001]: Stabilized group console-consumer-71906 generation 1 (__consumer_offsets-3 4) with 1 members (kafka.coordinator.group.GroupCoordinator) 2023-05-18 15:08:33 comments-kafka-1 [2023-05-18 22:08:33,906] INFO [GroupCoordi nator 1001]: Assignment received from leader for group console-consumer-71906 for gener ation 1. The group has 1 members, 0 of which are static. (kafka.coordinator.group.Group Coordinator) 2023-05-18 15:09:15 comments-kafka-1 [2023-05-18 22:09:15,466] INFO [Log partiti on=comments-0, dir=/kafka/kafka-logs-45d44a4a9d80] Deleting segments as the log has bee n deleted: LogSegment(baseOffset=0, size=2453323, lastModifiedTime=1684447415516, large stRecordTimestamp=Some(1684447415515)) (kafka.log.Log) 2023-05-18 15:09:15 comments-kafka-1 [2023-05-18 22:09:15,470] INFO [Log partiti on=comments-0, dir=/kafka/kafka-logs-45d44a4a9d80] Deleting segment files LogSegment(ba seOffset=0, size=2453323, lastModifiedTime=1684447415516, largestRecordTimestamp=Some(1 684447415515)) (kafka.log.Log) 2023-05-18 15:09:15 comments-kafka-1 [2023-05-18 22:09:15,483] INFO Deleted log /kafka/kafka-logs-45d44a4a9d80/comments-0.5c6f026100094955aaa733184ad0d973-delete/00000 00000000000000000000.log.deleted. (kafka.log.LogSegment) 2023-05-18 15:09:15 comments-kafka-1 [2023-05-18 22:09:15,488] INFO Deleted offs et index /kafka/kafka-logs-45d44a4a9d80/comments-0.5c6f026100094955aaa733184ad0d973-del ete/00000000000000000000000000000000.index.deleted. (kafka.log.LogSegment) 2023-05-18 15:09:15 comments-kafka-1 [2023-05-18 22:09:15,489] INFO Deleted time index /kafka/kafka-logs-45d44a4a9d80/comments-0.5c6f026100094955aaa733184ad0d973-delet e/00000000000000000000000000000000.timeindex.deleted. (kafka.log.LogSegment) 2023-05-18 15:09:15 comments-kafka-1 [2023-05-18 22:09:15,492] INFO Deleted log for partition comments-0 in /kafka/kafka-logs-45d44a4a9d80/comments-0.5c6f026100094955a aa733184ad0d973-delete. (kafka.log.LogManager)
comments-mongo-1 mongo:latest Running 27017:27017	
comments-kafka-1 wurstmeister/kafka Running 9092:9092	
comments-redis-1 redis:latest Running 6379:6379	



Running components by docker

You can install the command line tools to test them if you want:

- [Kafka cli tools](#)
- [Redis cli](#)
- [Mongosh](#) (mongodb cli)

Next, let's go to the **app folder** and install the pip dependencies

`pip install -r requirements.txt` (and yes, you can use a **virtualenv** if you want. I'm just simplifying things here).

Next, open another terminal window and start running the **comment_uploader** API server `python comment_uploader.py`. This will start the server at port 5051:

```
→ app python comment_uploader.py * Serving Flask app 'comment_uploader' * Debug
mode: off WARNING: This is a development server. Do not use it in a production
deployment. Use a production WSGI server instead. * Running on all addresses
(0.0.0.0) * Running on http://127.0.0.1:5001 * Running on
http://192.168.1.69:5001
```

Next, open another terminal window and start running the Kafka consumer using `python db_updater.py`. The app will simply start and wait for messages to come through.

```
→ app python db_updater.py
```

Finally, open another terminal window and start running the `comment_fetcher` API server that reads from Redis cache and returns comments: `python comment_fetcher.py` .

Optionally, you could monitor new messages by running Kafka's cli tool.

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic comments --from-beginning
```

Testing the system

Now, let's try sending a comment by hitting our write API:

```
curl -X POST -d '{"post_id":"1", "comment":"this is a great post thank you"}' -H "Content-Type: application/json" http://localhost:5001/comments
```

You should see the message propagating through our system.

1. Our write API at `localhost:5001/comments` should accept the request:

```
127.0.0.1 - - [18/May/2023 15:10:34] "POST /comments HTTP/1.1" 200 -
```

1. (optional) If you have Kafka cli running, you should see a new message in the log:

```
{"post_id": "1", "comment": "this is a great post thank you"}
```

1. `database_updater` should see the new comment from

```
→ app python db_updater.py {'post_id': '1', 'comment': 'this is a great post thank you', '_id': ObjectId('6466a681bec0bd23bec4ce12')}
```

1. We should see a new value in our database:

```
socialnetwork> db.comments.find() [ { _id: ObjectId("6466a681bec0bd23bec4ce12"), post_id: '1', comment: 'this is a great post thank you' } ]
```

1. Redis should have a new record

```
127.0.0.1:6379> hgetall "post:1" 1) "6466a681bec0bd23bec4ce12"
```

1. And finally, if we hit our user read API in `localhost:5002/comments?post_id=1` we should see the new comment:

```
{ "comments": [ { "_id": "6466a681bec0bd23bec4ce12", "comment": "this is a great post thank you", "post_id": "1" } ] }
```

Nice! It's working!

Handling Loads and Demonstrating Eventual consistency

Now, that's just one message. Let's send a ton of messages and see what happens. Create a file called `post_data.json` that contains the post body we just sent.

```
{"post_id":"1", "comment":"this is a great post thank you"}
```

And then, let's use [Apache Bench](#) to send 10000 messages! Here's the command:

```
ab -n 10000 -c 100 -p post_data.json -T 'application/json'
http://127.0.0.1:5001/comments
-n 10000 -c 100 means sending 100 messages each time until we've sent 10,000 messages.
```

You should see something like this output when you run the command:

```
→ app ab -n 10000 -c 100 -p post_data.json -T 'application/json'
http://127.0.0.1:5001/comments This is ApacheBench, Version 2.3 <$Revision:
1879490 $> Copyright 1996 Adam Twiss, Zeus Technology Ltd,
http://www.zeustech.net/ Licensed to The Apache Software Foundation,
http://www.apache.org/ Benchmarking 127.0.0.1 (be patient) Completed 1000
requests Completed 2000 requests Completed 3000 requests ...
```

And monitor your Redis content using `hgetall "post:1"` (or hit the read comment API at 5002) and you should see it returns fewer number of records than the number of requests. This is because there are too many messages for the consumer to handle, and there is a delay between the time a message is sent to Kafka and the time it is processed by the consumer. Eventually (after a few seconds), the consumer should catch up and Redis and the read comment API should return exactly the same results as in the MongoDB database. What we have demonstrated here are:

- Eventual consistency: When user A submits a new comment, we return 200 right away to tell the user the request is successful even though if user B fetches comments using the read API he may not necessarily see the user A's comment yet. There's an inconsistency in the system. The inconsistency disappears after a few seconds (eventually consistent). This is tolerable for a non-mission-critical application like social network.
- Message queue as a buffer: Without a message queue (Kafka), our service wouldn't be able to handle so many request as we cannot write to database fast enough. Kafka "buffers" the messages in-flight and allows us to scale the system.

As we will see, this template of separating the read and write path with message queue as a buffer will be used again and again in our system design journey and can be used to solve a variety of problems.